

ИТОГОВЫЙ КОНСПЕКТ: СПРИНТ ПО СОЗДАНИЮ ПРОТОТИПА LINGWARD

Версия 1.0

Автор: Кусов Владимир Михайлович

Дата: 27 августа 2026 г.

ЦЕЛЬ СПРИНТА

Создать рабочий прототип LINGWARD (CRM-ядро) с:

- Админкой Unfold (тёмная тема, фирменные цвета)
- Моделями: Репетиторы, Ученики, Учебные группы, Уроки, Отзывы
- Системой ролей (Manager, Tutor, Student, Administrator)
- Демонстрационными данными для быстрого знакомства с платформой

ОТСТУПЛЕНИЕ: КАК РАБОТАЕТ DJANGO (МЕТАФОРА)

1. Django — это «домостроительный комбинат»

Это инструмент, который умеет строить веб-приложения. У него есть свои стандарты, технологии и правила.

Важное ограничение: за один раз он может строить **только одно здание** на одной стройплощадке.

2. Команды Django — это «заказы» на строительство

Команда	Что делает	Метафора
<code>django-admin startproject <имя> .</code>	Создаёт проект (стройплощадку + проектный офис)	Вызываем ДСК, выделяем стройплощадку и ставим проектный офис
<code>python manage.py startapp <имя></code>	Создаёт приложение (здание)	Даём команду прорабу построить здание для конкретной задачи
<code>python manage.py runserver</code>	Запускает сервер	Открываем двери здания для посетителей
<code>python manage.py makemigrations</code>	Готовит чертежи для базы данных	Составляем план коммуникаций
<code>python manage.py migrate</code>	Строит коммуникации по чертежам	Прокладываем водопровод и электричество

3. Кто есть кто на стройплощадке

Файл	Метафора	Что делает
<code>manage.py</code>	Прораб	Управляет всем строительством: даёт команды, запускает процессы, контролирует сроки
<code>settings.py</code>	Генеральный план и чертежи	Хранит все настройки: размеры, материалы, подключения к внешним сетям
<code>urls.py</code>	Схема этажей и нумерация квартир	Показывает, по какому адресу (URL) находится какая комната (view)
<code>wsgi.py</code>	Инструкция для входной двери (турникет)	Как синхронно пропускать посетителей (запросы) через главный вход
<code>asgi.py</code>	Инструкция для домофона	Как асинхронно и через WebSockets общаться с посетителями
<code>models.py</code>	Планировка комнат	Описывает, какие данные и как будут храниться
<code>views.py</code>	Вентиляция и логика	Определяет, как обрабатываются запросы и что показывается на выходе
<code>admin.py</code>	Офис Управляющей компании (УК)	Настраивает интерфейс для сотрудников УК, чтобы они могли управлять данными через админку
<code>apps.py</code>	Паспорт здания	Настройка самого приложения (здания): его имя, регистрация в общем реестре

4. Сравнение WSGI и ASGI

Характеристика	WSGI	ASGI
Расшифровка	Web Server Gateway Interface	Asynchronous Server Gateway Interface
Тип	Синхронный	Асинхронный
Стандарт	2003 (PEP 333)	2015 (PEP 3333)
Поддержка WebSockets	❌ Нет	✅ Да
Поддержка HTTP/2	❌ Нет	✅ Да
Одновременные соединения	Ограничено количеством процессов/поток	Может обрабатывать тысячи соединений в одном потоке
Производительность	Хорошая для синхронных запросов	Высокая для I/O-операций и WebSockets

Характеристика	WSGI	ASGI
Для чего подходит	Классические веб-приложения, REST API	Чаты, уведомления, доски, трансляции
Примеры серверов	Gunicorn, uWSGI, mod_wsgi	Uvicorn, Daphne, Hypercorn
В проекте LINGWARD	Используется в <code>wsgi.py</code> для основного бэкенда (Django)	Используется в <code>asgi.py</code> для WebSocket-соединений (доска, уведомления)

АРХИТЕКТУРА LINGWARD

Наш проект — это «жилой массив»

Мы строим не одно здание, а несколько, каждое из которых решает свою задачу:

- `crm` — здание для управления репетиторами и учениками.
- `billing` — здание для финансов (будет позже).
- `videocall` — здание для видеозвонков (будет позже).
- `whiteboard` — здание для интерактивной доски (будет позже).

Почему мы переносим приложения в папку `apps/`?

Django по умолчанию строит все здания **прямо на стройплощадке**, рядом с проектным офисом. Если зданий несколько — стройплощадка превращается в хаос.

Решение: мы создаём **микрорайон** (`apps/`) и **перевозим** туда все готовые здания. Это даёт:

- **Чистоту** — на стройплощадке остаётся только проектный офис (`core/`) и прораб (`manage.py`).
- **Порядок** — все здания находятся в одном месте.
- **Масштабируемость** — можно легко добавить новое здание, просто положив его в `apps/`.

СТАДИЯ 1. ПОДГОТОВКА ОКРУЖЕНИЯ

1.1 Создание папки проекта

powershell

```
mkdir LINGWARD_ROOT
cd LINGWARD_ROOT
```

1.2 Создание виртуального окружения

powershell

```
python -m venv .venv
```

1.3 Активация виртуального окружения

powershell

```
.\.venv\Scripts\activate
```

 **Возможная ошибка:**

text

Невозможно загрузить файл .\.venv\Scripts\Activate.ps1, так как выполнение сценариев отключено в этой системе.

Исправление:

powershell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

(Ответить Y (Yes))

powershell

```
.\.venv\Scripts\activate
```

1.4 Проверка версии Python

powershell

```
python --version
```

Результат: Python 3.13.13 (или выше).



СТАДИЯ 2. УСТАНОВКА ЗАВИСИМОСТЕЙ

powershell

```
pip install django django-unfold psycopg2-binary python-dotenv
```



СТАДИЯ 3. СОЗДАНИЕ ПРОЕКТА И СТРУКТУРЫ ПАПОК

3.1 Создание Django-проекта

Что такое Django-проект и приложение?

Термин	Метафора	Что это
Django-проект	Проектный офис	Настройки всего сайта (<code>settings.py</code> , <code>urls.py</code>)
Django-приложение	Здание	Бизнес-логика (<code>models.py</code> , <code>views.py</code>)

Команды:

powershell

```
django-admin startproject lingward .  
python manage.py startapp crm
```

3.2 Создание структуры папок

Создаём папки:

powershell

```
mkdir core  
mkdir apps  
mkdir api  
mkdir services  
mkdir utils  
mkdir templates  
mkdir static
```

⚠ **Особенность PowerShell:** Команда `mkdir core apps api` не работает. Нужно создавать папки по отдельности.

Перемещаем файлы:

powershell

```
move crm apps\  
move lingward\settings.py core\  
move lingward\urls.py core\  
move lingward\wsgi.py core\  
move lingward\asgi.py core\  
rmdir lingward
```

3.3 Создание файлов `__init__.py`

powershell

```
New-Item core\__init__.py  
New-Item apps\__init__.py  
New-Item api\__init__.py  
New-Item services\__init__.py  
New-Item utils\__init__.py
```

Итоговая структура

text

```
LINGWARD_ROOT/  
├─ .venv/  
├─ manage.py  
├─ core/  
│   ├── __init__.py  
│   ├── settings.py  
│   ├── urls.py  
│   ├── wsgi.py  
│   └─ asgi.py  
├─ apps/  
│   ├── __init__.py  
│   └─ crm/  
│       ├── __init__.py  
│       ├── admin.py  
│       ├── apps.py  
│       ├── models.py  
│       └─ views.py  
├─ api/  
│   └─ __init__.py  
├─ services/  
│   └─ __init__.py  
├─ utils/  
│   └─ __init__.py  
├─ templates/  
├─ static/  
└─ db.sqlite3
```

СТАДИЯ 4. НАСТРОЙКА ПЕРЕМЕННЫХ ОКРУЖЕНИЯ И `settings.py`

4.1 Создание файла `.env`

Файл: `.env` (в корне проекта, рядом с `manage.py`)

Содержимое:

`env`

```
SECRET_KEY=ваш-уникальный-секретный-ключ
DEBUG=True
```

Как получить `SECRET_KEY`:

Выполните в терминале:

powershell

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

Скопируйте полученный ключ и вставьте его в файл `.env` вместо `ваш-уникальный-секретный-ключ`.

 **Важно о `SECRET_KEY`:**

- Ключ, приведённый в конспекте, — **учебный пример**. Он **небезопасен** и не должен использоваться в реальных проектах.
- **Для локальной разработки** можно использовать ключ из конспекта или тот, что сгенерировал Django.
- **Для публикации кода на GitHub** и тем более для **продакшена** ключ **обязательно** должен быть уникальным и храниться в переменных окружения (файл `.env`).

Как поступить студентам, скачавшим проект с GitHub:

1. В репозитории есть файл `.env.example` (или описание в конспекте).
2. Студент копирует его в `.env` и генерирует свой уникальный ключ.
3. Вставляет полученный ключ в файл `.env`.

Запомните: `SECRET_KEY` — это «ключ от всех дверей» в Django. Если он попадёт в открытый доступ, злоумышленник сможет подделывать сессии и CSRF-токены. Никогда не публикуйте его на GitHub и не используйте публичные ключи в продакшене.

4.2 Настройка `core/settings.py`

Файл: `core/settings.py`

python

```
import os
from pathlib import Path
from dotenv import load_dotenv

# Загружаем переменные из .env
load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.getenv('SECRET_KEY')
DEBUG = os.getenv('DEBUG', 'False') == 'True'
ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'unfold',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'apps.crm',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'core.urls'
WSGI_APPLICATION = 'core.wsgi.application'
ASGI_APPLICATION = 'core.asgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
    },
]
```



```

        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

```

```

LANGUAGE_CODE = 'ru-ru'
TIME_ZONE = 'Europe/Moscow'
USE_I18N = True
USE_TZ = True

```

```

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles'

```

```

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

```

```

UNFOLD = {
    "SITE_TITLE": "LINGWARD CRM",
    "SITE_HEADER": "Управление платформой",
    "SITE_URL": "/",
    "THEME": "dark",
    "COLORS": {
        "primary": {
            "50": "250, 245, 255",
            "100": "243, 232, 255",
            "200": "229, 211, 255",
            "300": "209, 186, 255",
            "400": "184, 158, 255",
            "500": "108, 92, 231",
            "600": "88, 72, 211",
            "700": "68, 52, 191",
            "800": "48, 32, 171",
            "900": "28, 12, 151",
        },
        "secondary": {
            "50": "240, 255, 255",
            "100": "220, 255, 255",
            "200": "180, 255, 255",
            "300": "140, 255, 255",
            "400": "100, 255, 255",
            "500": "0, 206, 201",
            "600": "0, 186, 181",
            "700": "0, 166, 161",
            "800": "0, 146, 141",
            "900": "0, 126, 121",
        },
    },
},
}

```

```
AUTH_USER_MODEL = 'crm.User'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

СТАДИЯ 5. ИСПРАВЛЕНИЕ ПУТЕЙ В `manage.py`, `wsgi.py`, `asgi.py`

Причина правки: Мы перенесли файлы из `lingward/` в `core/`, и Django должен знать, где теперь искать настройки.

Что нужно изменить в трёх файлах:

Было:

python

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'lingward.settings')
```

Стало:

python

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
```

Файлы для редактирования:

1. `manage.py` (в корне)
2. `core/wsgi.py` (после перемещения)
3. `core/asgi.py` (после перемещения)

СТАДИЯ 6. НАСТРОЙКА ПРИЛОЖЕНИЯ `apps.crm`

6.1 `apps/crm/apps.py`

python

```
from django.apps import AppConfig

class CrmConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'apps.crm'
    verbose_name = 'CRM'
```

6.2 apps/crm/__init__.py

python

```
default_app_config = 'apps.crm.apps.CrmConfig'
```



СТАДИЯ 7. СОЗДАНИЕ МОДЕЛЕЙ

Файл: apps/crm/models.py

python

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from django.core.validators import MinValueValidator, MaxValueValidator

# --- КОРТЕЖИ ДЛЯ ВЫБОРОВ ---
STATUS_CHOICES = (
    ('booked', 'Забронирован'),
    ('completed', 'Проведён'),
    ('cancelled', 'Отменён'),
    ('no_show', 'Неявка'),
)

LESSON_TYPE_CHOICES = (
    ('individual', 'Индивидуальный'),
    ('group', 'Групповой'),
)

class User(AbstractUser):
    ROLE_CHOICES = (
        ('tutor', 'Репетитор'),
        ('student', 'Ученик'),
        ('admin', 'Администратор'),
    )
    role = models.CharField(max_length=10, choices=ROLE_CHOICES,
default='student')
    phone = models.CharField(max_length=20, blank=True)
    is_active = models.BooleanField(default=True)

    def __str__(self):
        return self.username

    class Meta:
        verbose_name = "Пользователь"
        verbose_name_plural = "Пользователи"

class Tutor(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='tutor_profile')
```

```

    bio = models.TextField(blank=True)
    languages = models.JSONField(default=list)
    price_per_hour = models.DecimalField(max_digits=10, decimal_places=2,
default=0.00)
    rating = models.FloatField(default=0.0)
    is_verified = models.BooleanField(default=False)

    def __str__(self):
        return f"Репетитор: {self.user.username}"

    class Meta:
        verbose_name = "Репетитор"
        verbose_name_plural = "Репетиторы"

class Student(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='student_profile')
    level = models.CharField(max_length=2, blank=True)
    interests = models.JSONField(default=list)
    learning_goals = models.TextField(blank=True)

    def __str__(self):
        return f"Ученик: {self.user.username}"

    class Meta:
        verbose_name = "Ученик"
        verbose_name_plural = "Ученики"

class StudyGroup(models.Model):
    name = models.CharField(max_length=50, unique=True, verbose_name="Название
группы")
    tutor = models.ForeignKey(Tutor, on_delete=models.CASCADE,
related_name='study_groups')
    students = models.ManyToManyField(Student, related_name='study_groups',
verbose_name="Ученики")
    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Дата
создания")
    is_active = models.BooleanField(default=True, verbose_name="Активна")

    class Meta:
        verbose_name = "учебная группа"
        verbose_name_plural = "учебные группы"
        ordering = ['name']

    def __str__(self):
        return self.name

class Lesson(models.Model):
    tutor = models.ForeignKey(Tutor, on_delete=models.CASCADE,
related_name='lessons')
    lesson_type = models.CharField(max_length=10, choices=LESSON_TYPE_CHOICES,
default='individual', verbose_name="тип урока")

```

```

study_group = models.ForeignKey(StudyGroup, on_delete=models.SET_NULL,
null=True, blank=True, related_name='lessons', verbose_name="Учебная группа")
students = models.ManyToManyField(Student, related_name='lessons',
blank=True, verbose_name="ученики")
scheduled_at = models.DateTimeField(verbose_name="дата и время")
duration_minutes = models.IntegerField(default=60, verbose_name="длительность
(мин)")
status = models.CharField(max_length=10, choices=STATUS_CHOICES,
default='booked', verbose_name="Статус")
price = models.DecimalField(max_digits=10, decimal_places=2,
verbose_name="цена")
videocall_room_id = models.CharField(max_length=255, blank=True,
verbose_name="ID комнаты ВКС")
whiteboard_session_id = models.CharField(max_length=255, blank=True,
verbose_name="ID сессии доски")

class Meta:
    verbose_name = "Урок"
    verbose_name_plural = "Уроки"

def __str__(self):
    return f"Урок {self.tutor.user.username} → {self.study_group.name if
self.study_group else 'индивидуальный'}"

class Review(models.Model):
    lesson = models.OneToOneField(Lesson, on_delete=models.CASCADE,
related_name='review', verbose_name="Урок")
    rating = models.IntegerField(validators=[MinValueValidator(1),
MaxValueValidator(5)], verbose_name="Оценка")
    text = models.TextField(verbose_name="Текст отзыва")
    created_at = models.DateTimeField(auto_now_add=True, verbose_name="дата
создания")
    is_deleted = models.BooleanField(default=False, verbose_name="Удалён")

class Meta:
    verbose_name = "Отзыв"
    verbose_name_plural = "Отзывы"

def __str__(self):
    return f"Отзыв на урок #{self.lesson.id}"

```

СТАДИЯ 8. РЕГИСТРАЦИЯ МОДЕЛЕЙ В АДМИНКЕ

Файл: `apps/crm/admin.py`

python

```

from django.contrib import admin
from .models import Tutor, Student, StudyGroup, Lesson, Review

@admin.register(Tutor)

```

```

class TutorAdmin(admin.ModelAdmin):
    list_display = ('user', 'price_per_hour', 'rating', 'is_verified')
    list_filter = ('is_verified', 'languages')
    search_fields = ('user__username', 'user__first_name', 'user__last_name')
    ordering = ('-rating',)

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ('user', 'level')
    search_fields = ('user__username', 'user__first_name', 'user__last_name')

@admin.register(StudyGroup)
class StudyGroupAdmin(admin.ModelAdmin):
    list_display = ('name', 'tutor', 'is_active', 'created_at')
    list_filter = ('tutor', 'is_active')
    search_fields = ('name', 'tutor__user__username')
    filter_horizontal = ('students',)

@admin.register(Lesson)
class LessonAdmin(admin.ModelAdmin):
    list_display = ('tutor', 'study_group', 'scheduled_at', 'status', 'price')
    list_filter = ('status', 'scheduled_at')
    search_fields = ('tutor__user__username', 'study_group__name')

@admin.register(Review)
class ReviewAdmin(admin.ModelAdmin):
    list_display = ('lesson', 'rating', 'created_at')
    list_filter = ('rating', 'created_at')
    search_fields = ('lesson__tutor__user__username',
                    'lesson__study_group__name')

```



СТАДИЯ 9. МИГРАЦИИ И ЗАПУСК

9.1 Создание и применение миграций

powershell

```

python manage.py makemigrations
python manage.py migrate

```

9.2 Создание суперпользователя

powershell

```

python manage.py createsuperuser

```

Введите:

- **Имя пользователя:** `admin` (или любое другое)
- **Email:** `kvm64@yandex.ru` (или любой другой)
- **Пароль:** придумайте надёжный (не менее 8 символов)

9.3 Запуск сервера

powershell

```
python manage.py runserver
```

9.4 Проверка

Откройте браузер и перейдите по адресу: `http://127.0.0.1:8000/admin`

Вы должны увидеть **админку Unfold** с разделами:

- Репетиторы
- Ученики
- Учебные группы
- Уроки
- Отзывы

СТАДИЯ 10. ЗАПОЛНЕНИЕ ПЕРВОНАЧАЛЬНЫМИ ДАННЫМИ

Важное различие: `superuser` и бизнес-роль

- `superuser` — технический флаг, дающий полный доступ к серверу и настройкам Django.
- **Бизнес-роль** (`manager`, `Tutor`, `Student`, `Administrator`) — определяет действия пользователя на уровне школы.

Пример: Пользователь может быть `superuser` и иметь бизнес-роль `Student` — например, разработчик, тестирующий интерфейс ученика.

10.1 Создание системных ролей

Создайте папку `scripts/` в корне проекта.

Файл: `scripts/create_roles.py`

python

```
import os
import sys
import django

# добавляем путь к корню проекта
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
django.setup()

from django.contrib.auth.models import Group

def create_roles():
    print("🔧 Создание системных ролей...")
    roles = ['Manager', 'Tutor', 'Student', 'Administrator']
    for role in roles:
        group, created = Group.objects.get_or_create(name=role)
        if created:
            print(f"✅ Роль '{role}' создана.")
        else:
            print(f"❌ Роль '{role}' уже существует.")
    print("🎉 Системные роли созданы!")

if __name__ == '__main__':
    create_roles()

```

Запуск:

powershell

```
python scripts/create_roles.py
```

10.2 Создание демонстрационных данных

Файл: `scripts/create_demo_data.py`

python

```

import os
import sys
import django

# Добавляем путь к корню проекта
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
django.setup()

from django.contrib.auth import get_user_model
from apps.crm.models import Tutor, Student, StudyGroup, Lesson, Review
from decimal import Decimal
from datetime import datetime, timedelta

User = get_user_model()

def create_demo_data():
    print("🚀 Начинаем создание демонстрационных данных...")

```



```

# --- 1. МЕНЕДЖЕР ---
manager, _ = User.objects.get_or_create(
    username='manager',
    defaults={
        'email': 'manager@lingward.online',
        'first_name': 'Менеджер',
        'role': 'admin',
        'is_active': True,
        'is_staff': True,
        'is_superuser': True,
    }
)
manager.set_password('manager123')
manager.save()
print("✅ Менеджер создан.")

# --- 2. РЕПЕТИТОРЫ ---
tutor1_user, _ = User.objects.get_or_create(
    username='anna.petrova',
    defaults={
        'email': 'anna@lingward.online',
        'first_name': 'Анна',
        'last_name': 'Петрова',
        'role': 'tutor',
        'is_active': True,
    }
)
tutor1_user.set_password('tutor123')
tutor1_user.save()

tutor1, _ = Tutor.objects.get_or_create(
    user=tutor1_user,
    defaults={
        'bio': 'Репетитор английского языка. Опыт 5 лет.',
        'languages': ['Английский'],
        'price_per_hour': Decimal('2500.00'),
        'rating': 4.8,
        'is_verified': True,
    }
)
print("✅ Анна Петрова создана.")

tutor2_user, _ = User.objects.get_or_create(
    username='ivan.smirnov',
    defaults={
        'email': 'ivan@lingward.online',
        'first_name': 'Иван',
        'last_name': 'Смирнов',
        'role': 'tutor',
        'is_active': True,
    }
)
tutor2_user.set_password('tutor123')
tutor2_user.save()

tutor2, _ = Tutor.objects.get_or_create(

```

```

        user=tutor2_user,
        defaults={
            'bio': 'Репетитор французского языка. Носитель.',
            'languages': ['Французский'],
            'price_per_hour': Decimal('3000.00'),
            'rating': 4.5,
            'is_verified': True,
        }
    )
    print("✅ Иван Смирнов создан.")

# --- 3. УЧЕНИКИ (ГРУППА АННЫ) ---
group_students = []
group_names = [
    ('elena.egorova', 'Елена', 'Егорова'),
    ('mikhail.sidorov', 'Михаил', 'Сидоров'),
    ('olga.borisova', 'Ольга', 'Борисова'),
    ('alexey.vasiliev', 'Алексей', 'Васильев'),
    ('maria.grigorieva', 'Мария', 'Григорьева'),
]

for username, first, last in group_names:
    user, _ = User.objects.get_or_create(
        username=username,
        defaults={
            'email': f'{username}@lingward.online',
            'first_name': first,
            'last_name': last,
            'role': 'student',
            'is_active': True,
        }
    )
    user.set_password('student123')
    user.save()

    student, _ = Student.objects.get_or_create(
        user=user,
        defaults={
            'level': 'B1',
            'interests': ['разговорный', 'грамматика'],
            'learning_goals': 'Свободное владение английским',
        }
    )
    group_students.append(student)
    print(f"✅ {first} {last} создан(a).")

# --- 4. УЧЕБНАЯ ГРУППА ---
study_group, _ = StudyGroup.objects.get_or_create(
    name='Group-EN-01',
    defaults={
        'tutor': tutor1,
        'is_active': True,
    }
)
study_group.students.set(group_students)
print("✅ Учебная группа Group-EN-01 создана.")

```

```

# --- 5. ИНДИВИДУАЛЬНЫЕ УЧЕНИКИ (ИВАН) ---
individual_students = []
individual_names = [
    ('dmitry.grishin', 'Дмитрий', 'Гришин'),
    ('anna.davydova', 'Анна', 'Давыдова'),
    ('sergey.dmitriev', 'Сергей', 'Дмитриев'),
    ('ekaterina.fedorova', 'Екатерина', 'Фёдорова'),
    ('andrey.kuznetsov', 'Андрей', 'Кузнецов'),
]

for username, first, last in individual_names:
    user, _ = User.objects.get_or_create(
        username=username,
        defaults={
            'email': f'{username}@lingward.online',
            'first_name': first,
            'last_name': last,
            'role': 'student',
            'is_active': True,
        }
    )
    user.set_password('student123')
    user.save()

    student, _ = Student.objects.get_or_create(
        user=user,
        defaults={
            'level': 'A2',
            'interests': ['путешествия', 'кулинария'],
            'learning_goals': 'Общение в поездках',
        }
    )
    individual_students.append(student)
    print(f"✅ {first} {last} создан(a).")

# --- 6. УРОКИ ---
now = datetime.now().replace(hour=10, minute=0, second=0, microsecond=0)

lesson1, _ = Lesson.objects.get_or_create(
    tutor=tutor1,
    study_group=study_group,
    scheduled_at=now + timedelta(days=0),
    defaults={
        'lesson_type': 'group',
        'duration_minutes': 90,
        'status': 'booked',
        'price': Decimal('2500.00'),
    }
)
print(f"✅ Групповой урок создан.")

lesson2, _ = Lesson.objects.get_or_create(
    tutor=tutor1,
    study_group=study_group,
    scheduled_at=now + timedelta(days=1),

```

```

        defaults={
            'lesson_type': 'group',
            'duration_minutes': 90,
            'status': 'booked',
            'price': Decimal('2500.00'),
        }
    )
    print("✅ Второй групповой урок создан.")

    lesson3, _ = Lesson.objects.get_or_create(
        tutor=tutor2,
        scheduled_at=now + timedelta(hours=2),
        defaults={
            'lesson_type': 'individual',
            'duration_minutes': 60,
            'status': 'completed',
            'price': Decimal('3000.00'),
        }
    )
    lesson3.students.set([individual_students[0]])
    print("✅ Индивидуальный урок (проведён) создан.")

    lesson4, _ = Lesson.objects.get_or_create(
        tutor=tutor2,
        scheduled_at=now + timedelta(days=1, hours=4),
        defaults={
            'lesson_type': 'individual',
            'duration_minutes': 60,
            'status': 'booked',
            'price': Decimal('3000.00'),
        }
    )
    lesson4.students.set([individual_students[1]])
    print("✅ Индивидуальный урок (забронирован) создан.")

# --- 7. ОТЗЫВЫ ---
    review1, _ = Review.objects.get_or_create(
        lesson=lesson3,
        defaults={
            'rating': 5,
            'text': 'Отличный урок! Иван очень понятно объясняет.',
            'is_deleted': False,
        }
    )
    print("✅ Отзыв создан.")

    print("🎉 Демонстрационные данные успешно созданы!")

if __name__ == '__main__':
    create_demo_data()

```

Запуск:

powershell

```
python scripts/create_demo_data.py
```

10.3 Удаление демонстрационных данных

Файл: `scripts/delete_demo_data.py`

python

```
import os
import sys
import django

# Добавляем путь к корню проекта
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
django.setup()

from django.contrib.auth import get_user_model
from apps.crm.models import Tutor, Student, StudyGroup, Lesson, Review

User = get_user_model()

def delete_demo_data():
    print("🔧 начинаем удаление демонстрационных данных...")

    # --- 1. ОТЗЫВЫ ---
    Review.objects.filter(
        lesson__tutor__user__username__in=['anna.petrova', 'ivan.smirnov']
    ).delete()
    print("✅ Отзывы удалены.")

    # --- 2. УРОКИ ---
    Lesson.objects.filter(
        tutor__user__username__in=['anna.petrova', 'ivan.smirnov']
    ).delete()
    print("✅ Уроки удалены.")

    # --- 3. УЧЕБНЫЕ ГРУППЫ ---
    StudyGroup.objects.filter(
        tutor__user__username__in=['anna.petrova', 'ivan.smirnov']
    ).delete()
    print("✅ Учебные группы удалены.")

    # --- 4. СТУДЕНТЫ ---
    Student.objects.filter(
        user__username__in=[
            'elena.egorova', 'mikhail.sidorov', 'olga.borisova',
            'alexey.vasiliev', 'maria.grigorieva',
            'dmitry.grishin', 'anna.davydova', 'sergey.dmitriev',
            'ekaterina.fedorova', 'andrey.kuznetsov'
        ]
    ).delete()
```

```

print("✅ Студенты удалены.")

# --- 5. РЕПЕТИТОРЫ ---
Tutor.objects.filter(
    user__username__in=['anna.petrova', 'ivan.smirnov']
).delete()
print("✅ Репетиторы удалены.")

# --- 6. ПОЛЬЗОВАТЕЛИ (кроме менеджера) ---
User.objects.filter(
    username__in=[
        'anna.petrova', 'ivan.smirnov',
        'elena.egorova', 'mikhail.sidorov', 'olga.borisova',
        'alexey.vasiliev', 'maria.grigorieva',
        'dmitry.grishin', 'anna.davydova', 'sergey.dmitriev',
        'ekaterina.fedorova', 'andrey.kuznetsov'
    ]
).delete()
print("✅ Пользователи удалены.")

print("🎉 Демонстрационные данные успешно удалены!")

if __name__ == '__main__':
    delete_demo_data()

```

Запуск:

powershell

```
python scripts/delete_demo_data.py
```

ОШИБКИ И ИХ ИСПРАВЛЕНИЯ

1. Ошибка: `No module named 'lingward'`

Причина: В `manage.py`, `wsgi.py` и `asgi.py` указан старый путь к настройкам (`lingward.settings`).

Исправление: Замените во всех трёх файлах:

python

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'lingward.settings')
```

на:

python

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
```

2. Ошибка: No module named 'crm'

Причина: В `INSTALLED_APPS` в `settings.py` указано `'crm'`, а не `'apps.crm'`.

Исправление: В `core/settings.py` измените:

python

```
INSTALLED_APPS = [  
    ...  
    'crm',  
]
```

на:

python

```
INSTALLED_APPS = [  
    ...  
    'apps.crm',  
]
```

3. Ошибка: Cannot import 'crm'

Причина: В `apps/crm/apps.py` указано `name = 'crm'`, а нужно `name = 'apps.crm'`.

Исправление: В `apps/crm/apps.py` измените:

python

```
class CrmConfig(AppConfig):  
    name = 'crm'
```

на:

python

```
class CrmConfig(AppConfig):  
    name = 'apps.crm'
```

4. Ошибка: 'str' object has no attribute 'items'

Причина: Неправильный формат цветов в `UNFOLD`.

Исправление: Использовать словарь с оттенками (как в Стадии 4).

5. Ошибка: InconsistentMigrationHistory

Причина: Конфликт между стандартной моделью `auth.User` и кастомной `crm.User`.

Исправление:

1. Удалите старую базу данных:

powershell

```
del db.sqlite3
```

2. Удалите папку с миграциями `apps/crm/migrations` (кроме `__init__.py`) и создайте её заново:

powershell

```
Remove-Item -Recurse -Force apps\crm\migrations
mkdir apps\crm\migrations
New-Item apps\crm\migrations\__init__.py
```

3. Выполните миграции заново:

powershell

```
python manage.py makemigrations
python manage.py migrate
```

6. Ошибка: Не удастся загрузить файл Activate.ps1

Причина: В PowerShell по умолчанию отключено выполнение сценариев.

Исправление: Выполните команду и подтвердите действие:

powershell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

После этого активируйте окружение:

powershell


```
..\venv\Scripts\activate
```

7. Ошибка: `NameError: name 'LESSON_TYPE_CHOICES' is not defined`

Причина: Кортеж `LESSON_TYPE_CHOICES` определён внутри класса `Lesson`, а используется на уровне модуля.

Исправление: Вынесите определения `STATUS_CHOICES` и `LESSON_TYPE_CHOICES` на уровень модуля (в начало файла, после импортов).

8. Ошибка: Склеивание строк в `return f"Отзыв на урок #{self.lesson.id}"from django.db import models`

Причина: При копировании строка с импортом случайно приклеилась к `return`.

Исправление: Удалите лишний импорт в конце файла. В конце `models.py` должна остаться только строка:

python

```
return f"Отзыв на урок #{self.lesson.id}"
```

9. Ошибка: `ModuleNotFoundError: No module named 'dotenv'`

Причина: Библиотека `python-dotenv` не установлена.

Исправление:

powershell

```
pip install python-dotenv
```

10. Ошибка: `error: src refspec main does not match any`

Причина: Локальная ветка называется `master`, а не `main`.

Исправление:

powershell

```
git branch -m master main
git push -u origin main
```

11. Ошибка: ! [rejected] main -> main (fetch first)

Причина: В удалённом репозитории уже есть ветка `main` со старыми коммитами.

Исправление (если вы точно хотите перезаписать):

powershell

```
git push -f origin main
```



ПРИЛОЖЕНИЕ: ФАЙЛ `.gitignore`

gitignore

```
# Виртуальное окружение
.venv/
venv/
env/

# База данных
*.sqlite3
db.sqlite3

# файлы с секретами
.env
.env.local
.env.*.local

# коммерческие файлы (White Label)
*-wl.py
*-wl.js
*-wl.css
*-wl.html
config/*-wl.py

# логи
*.log

# кэш Python
__pycache__/*
*.pyc
*.pyo

# файлы IDE
```

```
.vscode/  
.idea/  
*.swp  
*.swo  
  
# Скомпилированные файлы  
*.pyd  
*.so  
*.dll  
  
# Статика и медиа (в продакшене собираются отдельно)  
staticfiles/  
media/
```

ИТОГ СПРИНТА

В результате мы получили:

- ☒ Работающий Django-проект с админкой Unfold (тёмная тема, фирменные цвета).
- ☒ Модели: Репетиторы, Ученики, Учебные группы, Уроки, Отзывы.
- ☒ Систему ролей (Manager, Tutor, Student, Administrator).
- ☒ Демонстрационные данные для быстрого знакомства с платформой.
- ☒ Готовность к дальнейшему расширению (API, провайдеры, фронтенд).
- ☒ Проект выложен на GitHub и готов к использованию студентами.

Сделано с ❤️ для образования и развития.

Кусов Владимир Михайлович, 27 августа 2026 года.